

Introdução à Teoria da Computação

ACH2043 — Resumo de Estudo para a Prova

Linguagens Livres-do-Contexto: Gramáticas, Forma Normal de Chomsky, Algoritmo CYK, Autômatos com Pilha e Equivalência $AP \leftrightarrow GLC$

Sumário

1	Aula 10 — Gramáticas Livres-do-Contexto (Parte 1) pág. 2
	Conceitos, derivação, árvores sintáticas e definição formal
2	Aula 11 — Construção de GLCs e Ambiguidade pág. 7
	Estratégias de projeto, conversões e ambiguidade
3	Aula 12 — Forma Normal de Chomsky pág. 15
	Os quatro passos de conversão e as propriedades da FNC
4	Aula 13 — Algoritmo CYK pág. 22
	A matriz triangular e o reconhecimento em tempo polinomial
5	Aula 14 — Autômato com Pilha pág. 29
	Definição formal e coleção de exemplos com diagramas
6	Equivalência $AP \leftrightarrow GLC$ (Parte 1) — GLC para AP pág. 36
	Construção de um autômato com pilha a partir de uma gramática
7	Equivalência $AP \leftrightarrow GLC$ (Parte 2) — AP para GLC pág. 42
	Construção de uma gramática a partir de um autômato com pilha

Resumo de Estudo — Aula 10

Gramáticas Livres-do-Contexto (Parte 1) • Seção 2.1

Esta é a primeira aula sobre **gramáticas livres-do-contexto** (GLC). O objetivo é apresentar uma forma mais poderosa de descrever linguagens do que as ferramentas do Capítulo 1 (autômatos finitos e expressões regulares). Vamos partir do zero: o que é uma gramática, como ela gera cadeias, como representar isso por árvores, e como definir tudo formalmente.

1. Por que precisamos de gramáticas?

No Capítulo 1 estudamos as **linguagens regulares**, descritas por autômatos finitos e expressões regulares. Vimos, porém, que existem linguagens muito simples que **não** são regulares. O exemplo clássico é:

$$L = \{ 0^n 1^n \mid n \geq 0 \}$$

exemplos: ϵ , 01, 0011, 000111, ...

A intuição de por que essa linguagem não é regular: para aceitar $0^n 1^n$ seria preciso **contar** quantos 0s apareceram à esquerda e comparar com o número de 1s à direita. Como as cadeias podem ser arbitrariamente grandes, um autômato finito — que só tem uma quantidade *finita* de estados — não consegue guardar essa contagem ilimitada. (A prova formal usa o *Lema do Bombeamento*, da Seção 1.4.)

Ideia central da aula

As GLCs conseguem descrever linguagens como $0^n 1^n$ que estão **além** do alcance das linguagens regulares. Toda linguagem regular é livre-do-contexto, mas nem toda livre-do-contexto é regular — ou seja, as GLCs formam uma classe estritamente maior.

2. De onde vem a ideia: gramáticas de línguas

O conceito de gramática nasceu no estudo da estrutura de frases em línguas humanas. Considere a frase «o rato roeu a roupa do rei de Roma». Nem toda justaposição das mesmas palavras faz sentido («rato rei de roupa a o roeu Roma do»). Para uma frase ter significado precisamos de dois ingredientes:

- **Elementos léxicos** — as palavras/símbolos válidos do idioma ("o", "rato", "roeu"...).
- **Estrutura (sintaxe)** — como esses elementos se combinam recursivamente: [o rato] [roeu a roupa [do rei de Roma]].

Uma **gramática** é justamente o conjunto de regras que define a estrutura sintática das frases a partir de seus elementos léxicos. Em computação, isso é a base da **análise sintática** de linguagens de programação (a 2ª das cinco fases de um compilador: análise léxica, **sintática**, semântica, otimização e geração de código).

3. Os componentes de uma gramática

Uma gramática consiste de uma coleção de **regras de substituição** (ou de **produção**) na forma $V \rightarrow x$, onde V é uma variável e x é uma cadeia formada por variáveis e/ou símbolos terminais. Vamos ver o vocabulário com o exemplo da gramática G1:

Gramática G1

$$A \rightarrow 0 A 1$$
$$A \rightarrow B$$
$$B \rightarrow \#$$

Vocabulário fundamental (decore isto!)

- **Variáveis** (ou símbolos **não-terminais**): representadas por letras maiúsculas — aqui A e B. São "placeholders" que ainda serão substituídos.
- **Símbolos terminais**: o alfabeto de saída, em minúsculas/dígitos/símbolos especiais — aqui 0, 1 e #. São o que sobra no final, quando não há mais o que substituir.
- **Variável inicial**: por onde toda derivação começa. Por convenção, é a variável do lado esquerdo da **primeira** regra — aqui, A.
- **Regras de substituição/produção**: as setas $V \rightarrow x$ que dizem como trocar uma variável por uma cadeia.

4. Gerando cadeias: o processo de derivação

Uma gramática **gera** cadeias de uma linguagem seguindo um procedimento simples:

- **Passo 1.** Escreva a variável inicial.
- **Passo 2.** Escolha uma variável V presente na cadeia atual, ache uma regra $V \rightarrow w$ e substitua V pelo lado direito w .
- **Passo 3.** Repita o passo 2 até não sobrar nenhuma variável.

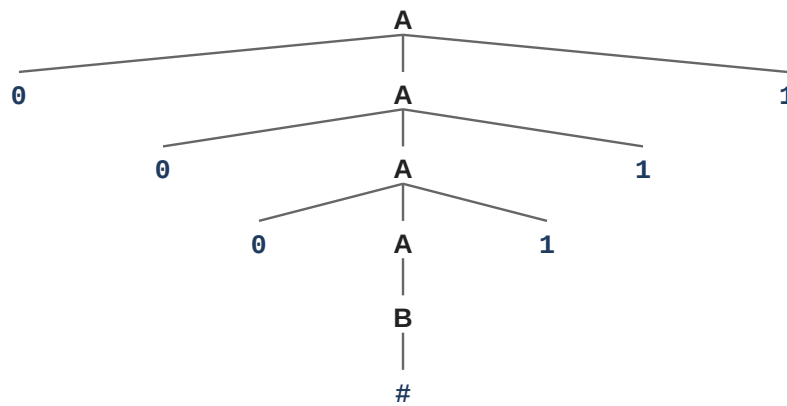
A sequência de substituições até chegar a uma cadeia só de terminais chama-se **derivação**. Veja a geração da cadeia **000#111** por G1:

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

Em cada passo, a regra usada foi, respectivamente: $A \rightarrow 0A1$ (três vezes), depois $A \rightarrow B$ e por fim $B \rightarrow \#$. Quando não há mais variáveis, a derivação termina.

A mesma derivação como árvore sintática

A derivação também pode ser desenhada como uma **árvore sintática** (parse tree): a raiz é a variável inicial, cada nó interno é uma variável e seus filhos são o lado direito da regra aplicada. Lendo as folhas da esquerda para a direita, recuperamos a cadeia gerada. Essa forma é mais clara que a lista de passos porque mostra a **estrutura** e não a ordem das substituições:



Árvore sintática de 000#111 em G1. As folhas, lidas da esquerda para a direita, formam a cadeia.

5. A linguagem gerada por uma gramática

A **linguagem de uma gramática G**, escrita $L(G)$, é o conjunto de todas as cadeias (só de terminais) que G consegue gerar. Toda linguagem que possa ser gerada por alguma GLC é chamada **linguagem livre-do-contexto (LLC)**.

Qual é a linguagem de G1? Gerando algumas cadeias — #, 0#1, 00#11, 000#111 — percebemos o padrão: um bloco de n zeros, um #, e um bloco de n uns. Portanto:

$$L(G1) = \{ 0^n \# 1^n \mid n \geq 0 \}$$

6. Definição formal de uma GLC

Definição — Gramática Livre-do-Contexto

Uma gramática livre-do-contexto é uma **4-upla** (V, Σ, R, S) , onde:

- V é um conjunto finito de **variáveis**;
- Σ (sigma) é um conjunto finito de **terminais**, disjunto de V (Σ é o alfabeto);
- R é um conjunto finito de **regras**, cada uma da forma $V \rightarrow$ (cadeia de variáveis e terminais);
- $S \in V$ é a **variável inicial**.

Exemplo — definição formal de G1: $V = \{A, B\}$, $\Sigma = \{0, 1, \#\}$, $S = A$, e R são as três regras já vistas. Na prática, muitas vezes especificamos uma gramática escrevendo **apenas as regras**; nesse caso, as variáveis são os símbolos que aparecem à esquerda das setas, os terminais são todos os outros, e a variável inicial é a da primeira regra.

Notação de derivação (precisa)

Sejam u, v, w cadeias de variáveis e terminais e $A \rightarrow v$ uma regra. Dizemos que **uAw origina uvw** , escrito $uAw \Rightarrow uvw$ (uma derivação direta, um único passo). Dizemos que **u deriva v** , escrito $u \Rightarrow^* v$, se $u = v$ ou se existe uma sequência de passos $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow v$.

Com essa notação, a linguagem de G é definida formalmente como:

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

7. Mais exemplos de GLCs

Exemplo — parênteses aninhados (G3). Seja $V = \{S\}$, $\Sigma = \{0, 1, (,)\}$, S inicial, e a única regra (em notação compacta, com $|$ separando alternativas):

Gramática G3

$$S \rightarrow (S) \mid S S \mid 0 \mid 1$$

Algumas cadeias geradas: $0, 1, (000), (1(0)), ((11)1((1)01))$. $L(G3)$ = cadeias binárias não vazias com parênteses aninhados. Note como a regra $S \rightarrow (S)$ permite **recursão**, gerando aninhamentos de profundidade ilimitada.

Exemplo 2.4 — expressões aritméticas (G4). Uma gramática que descreve um fragmento da sintaxe de uma linguagem de programação:

Gramática G4

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{TERMO} \rangle \mid \langle \text{TERMO} \rangle$$

$$\langle \text{TERMO} \rangle \rightarrow \langle \text{TERMO} \rangle \times \langle \text{FATOR} \rangle \mid \langle \text{FATOR} \rangle$$

$$\langle \text{FATOR} \rangle \rightarrow (\langle \text{EXPR} \rangle) \mid a$$

$L(G4)$ = expressões aritméticas com $+$ e $\times$ sobre o operando a , incluindo parênteses (o que conta como operando válido no lugar de a é decidido pelo analisador léxico). O detalhe importante: G4 foi projetada para **respeitar a precedência** de $\times$ sobre $+$, e isso se reflete no formato único das árvores sintáticas de cada expressão — assunto que retomaremos ao falar de ambiguidade na próxima aula.

Exercícios resolvidos

Os exercícios abaixo consolidam os conceitos da aula. Onde a aula pede para «simular a derivação de algumas cadeias», resolvemos explicitamente.

Exercício 1 — Derivar 00#11 em G1

Aplicamos as regras a partir da variável inicial A, sempre substituindo a variável presente:

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 00B11 \Rightarrow 00\#11$$

Passos: $A \rightarrow 0A1$ (duas vezes) para criar dois 0s à esquerda e dois 1s à direita; depois $A \rightarrow B$; por fim $B \rightarrow \#$. A cadeia resultante 00#11 pertence a $L(G1)$ com $n = 2$. ✓

Exercício 2 — $L(G3)$ contém a cadeia (1(0))?

Precisamos exibir uma derivação. Lembre as regras: $S \rightarrow (S) \mid SS \mid 0 \mid 1$.

$$S \Rightarrow (S) \Rightarrow (SS) \Rightarrow (1S) \Rightarrow (1(S)) \Rightarrow (1(0))$$

Como existe derivação a partir de S, a cadeia $(1(0)) \in L(G3)$. ✓

Exercício 3 — Gerar $a + a \times a$ em G4 respeitando a precedência

Fazemos a derivação de cima para baixo. Como + está no topo da estrutura de <EXPR>, ele fica "por fora", e $\times$ fica "por dentro" (dentro de <TERMO>), o que garante que a multiplicação seja avaliada primeiro:

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \\ &\Rightarrow a + T \times F \Rightarrow a + F \times F \Rightarrow a + a \times F \Rightarrow a + a \times a \end{aligned}$$

(usamos E, T, F como abreviações de <EXPR>, <TERMO>, <FATOR>). O fato de a subexpressão $a \times a$ nascer agrupada dentro de um único <TERMO> é o que codifica a precedência correta. ✓

Exercício 4 — Qual a linguagem de $S \rightarrow aSb \mid \epsilon$?

Gerando cadeias: $S \Rightarrow \epsilon$; $S \Rightarrow aSb \Rightarrow ab$; $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$. Cada aplicação de $S \rightarrow aSb$ acrescenta simultaneamente um a à esquerda e um b à direita, mantendo-os em igual número. Logo:

$$L = \{ a^n b^n \mid n \geq 0 \}$$

Este é exatamente o tipo de linguagem não-regular que motivou as GLCs — a recursão "casada" da regra aSb faz o papel da contagem que o autômato finito não conseguia. ✓

Resumo de Estudo — Aula 11

Construção de GLCs e Ambiguidade • Seção 2.1

Na aula anterior aprendemos o que é uma gramática livre-do-contexto e como ela gera cadeias. Agora vamos aprender o lado prático — **como projetar** uma GLC para uma linguagem dada — e um problema importante que surge nesse processo: a **ambiguidade**, quando uma mesma cadeia pode ter mais de uma estrutura.

1. Projetando gramáticas livres-do-contexto

Projetar uma GLC exige certa criatividade, mas existem estratégias que quase sempre funcionam. A dica geral para testar uma gramática que você propôs é **simular a derivação de algumas cadeias** e verificar se batem com a linguagem desejada. Vamos primeiro construir gramáticas para linguagens simples (com $\Sigma = \{a, b\}$) e depois extrair as estratégias gerais.

Bloco de gramáticas básicas (memorize os padrões)

Estas são as "peças de LEGO" a partir das quais montamos gramáticas maiores:

- | | | | |
|----|-----------------------------|--|--|
| 1. | $L = \{a\}$ | $S \rightarrow a$ | |
| 2. | $L = \{\epsilon\}$ | $S \rightarrow \epsilon$ | |
| 3. | $L = \{a, b\}$ | $S \rightarrow a \mid b$ | |
| 4. | $L = a^*$ | $S \rightarrow aS \mid \epsilon$ | (ou $S \rightarrow Sa \mid \epsilon$) |
| 5. | $L = a^+$ | $S \rightarrow aS \mid a$ | (ou $S \rightarrow Sa \mid a$) |
| 6. | $L = \Sigma^* = \{a, b\}^*$ | $S \rightarrow aS \mid bS \mid \epsilon$ | |
| 7. | $L = \Sigma^+ = \{a, b\}^+$ | $S \rightarrow aS \mid bS \mid a \mid b$ | |

Observe o par de opções em 4 e 5: pôr a variável à direita (aS) constrói a cadeia da esquerda para a direita; pô-la à esquerda (Sa) constrói da direita para a esquerda. Ambas geram a mesma linguagem.

Linguagens um pouco mais elaboradas

$L = \{ \text{cadeias que começam e terminam com } a \}$

A expressão regular é $a\Sigma^*a \cup \{a\}$. A parte $a\Sigma^*a$ é um "aninhamento": um a fixo no início, um a fixo no fim, e qualquer coisa no meio (uma sub-gramática T que gera Σ^*). Adicionamos a regra $S \rightarrow a$ para cobrir a cadeia unitária «a»:

$$\begin{aligned} S &\rightarrow a T a \mid a \\ T &\rightarrow Ta \mid Tb \mid \epsilon \end{aligned}$$

$L = \{ \text{cadeias que começam e terminam com o mesmo símbolo} \}$

Aqui $L = L_1 \cup L_2$, onde L_1 começa/termina com a e L_2 com b. A estratégia da **união** é criar uma variável inicial que escolhe entre as duas:

$$\begin{aligned} S &\rightarrow S1 \mid S2 \\ S1 &\rightarrow a \mid a T a \\ S2 &\rightarrow b \mid b T b \\ T &\rightarrow Ta \mid Tb \mid \epsilon \end{aligned}$$

$L = \{ a^n b^n \mid n \geq 0 \}$ — a estratégia recursiva

Toda palavra não-vazia se escreve em função de uma menor: $w_n = a \cdot w_{n-1} \cdot b$, com caso base $w_0 = \epsilon$. Essa recursão vira diretamente uma regra que envolve a variável de si mesma, cercada pelos símbolos casados:

$$S \rightarrow a S b \mid \epsilon$$

$L = \{ a^m b^m c^n d^n \mid m, n \geq 0 \}$ — concatenação

$L = L_1 \cdot L_2$ (concatenação), com $L_1 = \{ a^m b^m \}$ e $L_2 = \{ c^n d^n \}$. Já sabemos construir cada uma; para concatenar, basta uma variável inicial que gera uma seguida da outra:

$$\begin{aligned} S &\rightarrow S1 S2 \\ S1 &\rightarrow a S1 b \mid \epsilon \\ S2 &\rightarrow c S2 d \mid \epsilon \end{aligned}$$

2. As quatro estratégias gerais

Generalizando os exemplos acima, temos um conjunto de técnicas que cobrem a maioria dos casos de projeto de GLCs:

Estratégia 1 — Composição de linguagens mais simples

LLCs costumam ser combinações de LLCs menores. Para a **união** $L_1 \cup L_2$: construa G_1 (variável inicial S_1) e G_2 (variável inicial S_2) separadamente, junte todas as regras e adicione $S \rightarrow S1 \mid S2$. Para a **concatenação** $L_1 \cdot L_2$: adicione $S \rightarrow S1 S2$.

Estratégia 2 — Toda linguagem regular tem uma GLC

Se você consegue um AFD/AFN ou uma expressão regular para a linguagem, a conversão para GLC é mecânica.

Convertendo um autômato (AFD/AFN) em GLC:

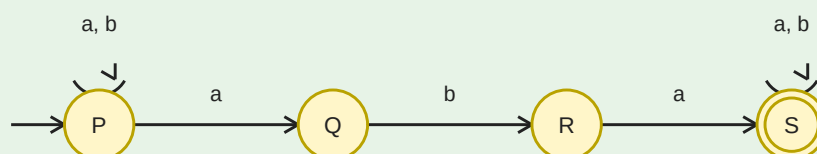
- Crie uma variável R_i para cada estado q_i .
- Para cada transição $q_i \xrightarrow{a} q_j$, adicione $R_i \rightarrow a R_j$; para transição- ϵ $q_i \xrightarrow{\epsilon} q_j$, adicione $R_i \rightarrow R_j$.
- Para cada estado de aceitação q_i , adicione $R_i \rightarrow \epsilon$.
- Torne R_0 (do estado inicial q_0) a variável inicial.

Convertendo uma expressão regular em GLC (recursivamente sobre a estrutura da ER):

$R = a$	$\rightarrow$	$S \rightarrow a$	(idem para ϵ)
$R = R1 \cup R2$	$\rightarrow$	$S \rightarrow S1 \mid S2$	
$R = R1 \cdot R2$	$\rightarrow$	$S \rightarrow S1 S2$	
$R = R1^*$	$\rightarrow$	$S \rightarrow S1 S \mid \epsilon$	
$R = R1^+$	$\rightarrow$	$S \rightarrow S1 S \mid S1$	

Aplicação — $L = \{ w \in \{a,b\}^* \mid w \text{ contém a subcadeia aba} \}$

Primeiro projetamos o autômato (AFN) que reconhece a linguagem: ele fica "parado" em P consumindo a's e b's, avança para Q ao ver o «a», para R ao ver «b», para S ao ver «a» (fechando o «aba»), e então aceita qualquer sufixo:



Aplicando a conversão estado→variável (P,Q,R,S viram variáveis; loops e transições viram regras; o estado de aceitação S ganha $S \rightarrow \epsilon$):

$$\begin{aligned} P &\rightarrow aP \mid bP \mid aQ \\ Q &\rightarrow bR \\ R &\rightarrow aS \\ S &\rightarrow aS \mid bS \mid \epsilon \end{aligned}$$

Note a correspondência direta: cada auto-loop de P vira $P \rightarrow aP$ e $P \rightarrow bP$; a transição $P \xrightarrow{a} Q$ vira $P \rightarrow aQ$; o estado final S produz a regra $S \rightarrow \epsilon$. (Há uma versão mais compacta via expressão regular, abaixo.)

Alternativamente, partindo da expressão regular $\Sigma^*aba\Sigma^*$, a conversão dá uma GLC bem mais enxuta para a mesma linguagem:

$$\begin{aligned} S &\rightarrow T a b a T \\ T &\rightarrow aT \mid bT \mid \epsilon \end{aligned}$$

Estratégia 3 — Subcadeias "ligadas" (correspondência)

Quando duas partes da cadeia precisam corresponder em quantidade ilimitada (uma máquina teria de "lembrar" quanto viu de uma para conferir a outra), use uma regra recursiva da forma $R \rightarrow u R v$. Ela gera camadas casadas de u e v de fora para dentro. Foi o que fizemos em $a^n b^n$ com $S \rightarrow aSb \mid \epsilon$.

Estratégia 4 — Estruturas recursivas aninhadas

Em linguagens complexas, uma estrutura pode reaparecer dentro de outra (ou de si mesma). Ex.: em expressões aritméticas, no lugar de um operando a pode surgir uma expressão parentizada inteira. Para conseguir isso, coloque a variável que gera a estrutura exatamente na posição onde ela pode reaparecer — como o <EXPR> dentro de <FATOR> $\rightarrow (\text{<EXPR>})$ na gramática G4.

3. Ambiguidade

Às vezes uma gramática gera a mesma cadeia de **várias maneiras estruturalmente diferentes** — ou seja, com árvores sintáticas distintas. Nesse caso dizemos que a cadeia é **derivada ambigualmente**, e que a gramática é **ambígua**. Como cada árvore corresponde a uma interpretação (a um "significado"), a ambiguidade é indesejável em linguagens de programação, onde um programa deve ter uma única leitura.

Definição precisa (cuidado com a pegadinha!)

Uma cadeia é gerada ambigualmente quando tem **duas árvores sintáticas diferentes** — **não** quando tem duas derivações diferentes. Duas derivações podem diferir só pela *ordem* em que substituem as variáveis, sem mudar a estrutura. Para eliminar essa distinção irrelevante, definimos um tipo padronizado de derivação (veja abaixo).

Derivação mais à esquerda

Uma **derivação mais à esquerda** é aquela em que, a cada passo, substituímos sempre a variável **mais à esquerda** da cadeia intermediária. Fixando essa ordem, cada árvore sintática corresponde a exatamente uma derivação mais à esquerda — então podemos comparar árvores comparando derivações mais à esquerda. Exemplo com G4, derivando $(a + a) \times a$:

$$\begin{aligned} E &\Rightarrow T \Rightarrow T \times F \Rightarrow F \times F \Rightarrow (E) \times F \Rightarrow (E + T) \times F \Rightarrow (T + T) \times F \\ &\Rightarrow (F + T) \times F \Rightarrow (a + T) \times F \Rightarrow (a + F) \times F \Rightarrow (a + a) \times F \Rightarrow (a + a) \times a \end{aligned}$$

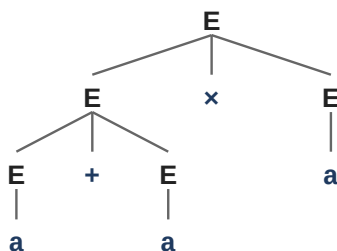
Exemplo clássico: G4 (não-ambígua) versus G5 (ambígua)

A gramática G4 da aula passada captura a precedência de $\times$ sobre $+$, e por isso cada expressão tem uma **única** árvore — logo, um único resultado. Já a gramática G5, abaixo, gera as mesmas expressões, porém de forma ambígua:

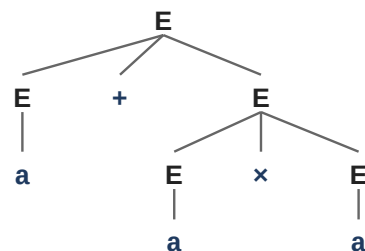
Gramática G5 (ambígua)

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$

A cadeia $a + a \times a$ tem duas árvores em G5. À esquerda, a estrutura agrupa $(a+a)$ primeiro e multiplica por a ; à direita, agrupa $(a \times a)$ primeiro e soma a . Se $a = 3$, a árvore da esquerda dá **18** e a da direita dá **12**:



Esquerda: $(a+a) \times a \rightarrow \text{resultado } 18$



Direita: $a + (a \times a) \rightarrow \text{resultado } 12$

Ambiguidade inerente

Muitas vezes uma gramática ambígua pode ser reescrita como uma gramática não-ambígua equivalente (foi o que G4 fez em relação a G5). Mas existem linguagens que **só** podem ser geradas por gramáticas ambíguas — são as **inerentemente ambíguas**. Exemplo:

$$L = \{ a^i b^j c^k \mid i = j \text{ ou } j = k, \quad i, j, k \geq 0 \}$$

Para a cadeia aaabbbccc não existe gramática com derivação única, porque ela satisfaz *ao mesmo tempo* $i = j$ e $j = k$, e qualquer gramática precisa "decidir" por qual das duas condições gerá-la. Note que L é a união de $\{a^i b^i c^*\}$ com $\{a^* b^j c^j\}$.

Ambiguidade em linguagens de programação: o problema do «dangling else»

Um caso famoso surge com a regra de comandos condicionais:

```
Statement → if Cond then Statement
           | if Cond then Statement else Statement
           | ...
```

A sentença «if a then if b then s1 else s2» pode ser lida de duas formas — o «else» pode pertencer ao primeiro ou ao segundo «if»:

- (1) if a then { if b then s1 } else s2
- (2) if a then { if b then s1 else s2 }

Formas de resolver: (a) **modificar a gramática** para torná-la não-ambígua — por exemplo, tornando um delimitador como «endif»/«fi» obrigatório; ou (b) manter a gramática e adotar uma **regra de desambiguação** — a mais comum é associar o «else» sempre ao «if» mais próximo (o último encontrado).

Exercícios resolvidos

Abaixo resolvemos os exercícios propostos nos slides. Onde havia apenas o enunciado, apresentamos a gramática e justificamos.

Exercício 2.4 — Dê GLCs para as linguagens abaixo ($\Sigma = \{0, 1\}$)

(a) $\{ w \mid w \text{ contém pelo menos três 1s} \}$

Basta forçar três 1s obrigatórios, separados por "lixo" arbitrário gerado por $X = \Sigma^*$:

$$\begin{aligned} S &\rightarrow X 1 X 1 X 1 X \\ X &\rightarrow 0X \mid 1X \mid \varepsilon \end{aligned}$$

(b) $\{ w \mid w \text{ começa e termina com o mesmo símbolo} \}$

Mesma ideia da união vista na aula (adaptada a $\{0,1\}$); inclui as cadeias unitárias 0 e 1:

$$\begin{aligned} S &\rightarrow 0 T 0 \mid 1 T 1 \mid 0 \mid 1 \\ T &\rightarrow 0T \mid 1T \mid \varepsilon \end{aligned}$$

(c) $\{ w \mid \text{o comprimento de } w \text{ é ímpar} \}$

Ideia: um símbolo central obrigatório (comprimento 1), e a possibilidade de acrescentar **dois** símbolos por vez à frente, o que mantém a paridade ímpar:

$$S \rightarrow 0 \mid 1 \mid 00S \mid 01S \mid 10S \mid 11S$$

Cada regra com dois símbolos à frente soma 2 ao comprimento (mantém ímpar); os casos base 0 e 1 dão comprimento 1. ✓

(d) $\{ w \mid \text{comprimento ímpar e o símbolo central é } 0 \}$

A variável S constrói pares casados de símbolos quaisquer em torno de um centro; o centro é forçado a ser 0:

$$S \rightarrow 0 S 0 \mid 0 S 1 \mid 1 S 0 \mid 1 S 1 \mid 0$$

A regra recursiva acrescenta um símbolo de cada lado (mantém o centro no meio) e o caso base $S \rightarrow 0$ fixa o símbolo central. ✓

(e) $\{ w \mid w = w^R, \text{ isto é, } w \text{ é um palíndromo} \}$

Um palíndromo se constrói de fora para dentro, adicionando o mesmo símbolo nas duas pontas; os casos base cobrem comprimento par (ε) e ímpar (0 ou 1):

$$S \rightarrow 0 S 0 \mid 1 S 1 \mid 0 \mid 1 \mid \varepsilon$$

(f) O conjunto vazio

Uma gramática cuja variável inicial não pode terminar nenhuma derivação (nunca chega a uma cadeia só de terminais) gera a linguagem vazia:

$$S \rightarrow S$$

Como S só pode ser trocado por S, nenhuma cadeia de terminais é jamais obtida, logo $L(G) = \emptyset$.
✓

Outros exercícios da aula

Adapte $S \rightarrow aSb \mid \epsilon$ para $L = \{ a^n b^{2n} \mid n \geq 0 \}$

Agora cada a precisa corresponder a **dois** b. Basta acrescentar dois b por aplicação da regra:

$$S \rightarrow a S b b \mid \epsilon$$

Derivando: $S \Rightarrow aSbb \Rightarrow aaSbbbb \Rightarrow aabbbb$ ($n=2$): a^2b^4 . ✓

Gramática para a linguagem inerentemente ambígua $\{ a^i b^j c^k \mid i=j \text{ ou } j=k \}$

Usando $L = \{ a^i b^j c^k \} \cup \{ a^* b^j c^j \}$, construímos uma sub-gramática para cada parte e unimos:

$$\begin{aligned} S &\rightarrow L1 \mid L2 \\ L1 &\rightarrow X C && (a^i b^i \text{ seguido de } c's) \\ X &\rightarrow a X b \mid \epsilon \\ C &\rightarrow c C \mid \epsilon \\ L2 &\rightarrow A Y && (a's \text{ seguido de } b^j c^j) \\ A &\rightarrow a A \mid \epsilon \\ Y &\rightarrow b Y c \mid \epsilon \end{aligned}$$

A cadeia aaabbbccc pode ser gerada tanto pelo ramo L1 ($i=j=3$, com C gerando ccc) quanto pelo ramo L2 ($j=k=3$, com A gerando aaa) — é exatamente esse duplo caminho que caracteriza a ambiguidade inerente. ✓

Mostre que «the girl touches the boy with the flower» é ambígua em G2

A ambiguidade está em **a quem o «with the flower» se liga**. Há duas leituras:

- **Leitura 1** — a flor é o *instrumento*: a garota, usando a flor, toca o menino. O «with the flower» (<PREP-PHRASE>) anexa-se ao <VERB-PHRASE> (junto do verbo «touches»).
- **Leitura 2** — a flor está *com o menino*: a garota toca o menino que está com a flor. O «with the flower» anexa-se ao <NOUN-PHRASE> «the boy».

As duas leituras correspondem a duas árvores sintáticas distintas (o <PREP-PHRASE> pendura em pontos diferentes da árvore), o que prova que G2 é ambígua. ✓

Resumo de Estudo — Aula 12

Forma Normal de Chomsky (FNC) • Seção 2.1

Já sabemos projetar gramáticas livres-do-contexto. Agora vamos aprender a colocá-las em uma **forma padronizada** — a Forma Normal de Chomsky — na qual todas as regras seguem um formato bem restrito. Essa padronização é a chave para criar algoritmos eficientes de análise sintática (como o CYK, da próxima aula) e para vários resultados teóricos.

1. O que é a Forma Normal de Chomsky

Ao trabalhar com GLCs é frequentemente conveniente tê-las numa forma simplificada. A mais simples e útil é a **Forma Normal de Chomsky (FNC)**. Suas utilidades principais: facilita projetar algoritmos de **análise sintática** (parsing) e serve de base para resultados de **computabilidade** (provar que toda LLC é decidível) e de **análise de complexidade**.

Definição 2.8 — Forma Normal de Chomsky

Uma GLC está na FNC se **todas** as suas regras têm um destes dois formatos (e mais duas condições sobre a variável inicial):

$V \rightarrow W X$	(duas variáveis)
$V \rightarrow a$	(um único terminal)

- Nas regras $V \rightarrow WX$, tanto W quanto X são **variáveis** (nunca terminais).
- Em $V \rightarrow a$, o lado direito é **um único terminal**.
- **(b)** A variável inicial S_0 não pode aparecer no lado **direito** de nenhuma regra.
- **(c)** Só a variável inicial pode ter a regra $S_0 \rightarrow \varepsilon$ (para permitir que a linguagem contenha a cadeia vazia).

*Em resumo: na FNC, cada passo de derivação ou junta duas variáveis, ou troca uma variável por um único terminal. Isso torna as árvores sintáticas **binárias**, o que é ótimo para algoritmos.*

2. Convertendo qualquer GLC para a FNC

Toda GLC pode ser transformada em uma equivalente na FNC (que gera exatamente a mesma linguagem). O procedimento tem **quatro passos**, e a cada passo a gramática continua equivalente à original:

Os quatro passos (nesta ordem)

- **Passo 1 — Nova variável inicial.** Adicione uma variável inicial nova S_0 e a regra $S_0 \rightarrow S$. Isso garante que a inicial nunca apareça à direita.
- **Passo 2 — Eliminar regras- ϵ** ($V \rightarrow \epsilon$, com $V \neq S_0$). Remova a regra e, para cada ocorrência de V no lado direito de outra regra, crie uma cópia da regra com essa ocorrência apagada.
- **Passo 3 — Eliminar regras unitárias** ($V \rightarrow U$, uma variável só). Remova a regra e copie para V todas as produções de U .
- **Passo 4 — Padronizar o restante** para os formatos $V \rightarrow WX$ ou $V \rightarrow a$, criando variáveis auxiliares onde necessário.

Passo 2 em detalhe (a parte mais delicada)

Ao eliminar $V \rightarrow \epsilon$: para **cada** ocorrência de V no lado direito de uma regra $U \rightarrow \dots V \dots$, adicione uma nova regra com essa ocorrência apagada. Se V aparece várias vezes na mesma regra, gere **todas** as combinações. Por exemplo, se $B \rightarrow uVvVw$, adicionam-se três regras:

$$B \rightarrow u \ v \ V \ w \quad | \quad u \ V \ v \ w \quad | \quad u \ v \ w$$

E se existir a regra $U \rightarrow V$ (só V à direita), adicione $U \rightarrow \epsilon$ — a menos que $U \rightarrow \epsilon$ já tenha sido removida antes (para não entrar em loop). A variável inicial S_0 é a única que pode manter $S_0 \rightarrow \epsilon$.

Passo 3 em detalhe

Ao eliminar a regra unitária $V \rightarrow U$: para toda regra $U \rightarrow u$, adicione $V \rightarrow u$ (a menos que seja uma unitária já removida). Regras da forma $V \rightarrow V$ são simplesmente descartadas (são redundantes). Repita até não sobrar nenhuma regra unitária.

Passo 4 em detalhe

Para regras longas $V \rightarrow u_1 u_2 \dots u_k$ com $k \geq 3$, quebre-as numa cadeia de regras binárias criando variáveis auxiliares $V_1, V_2, \dots$:

$$V \rightarrow u_1 \ V_1 \quad V_1 \rightarrow u_2 \ V_2 \quad V_2 \rightarrow u_3 \ V_3 \quad \dots \quad V_{(k-2)} \rightarrow u_{(k-1)} \ u_k$$

E qualquer terminal u_i que ainda apareça numa regra binária é substituído por uma nova variável U_i , com a regra $U_i \rightarrow u_i$.

3. Exemplo completo de conversão

Vamos converter a gramática abaixo passo a passo. Acompanhe como cada etapa modifica as regras — as versões intermediárias são G1, G2, ... até G8, a FNC final.

Gramática inicial G

$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

Passo 1 — nova variável inicial (adiciona $S_0 \rightarrow S$):

G1

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

Passo 2 — eliminar $B \rightarrow \varepsilon$. B aparece em «aB» (vira «a») e em «A → B» (vira A → ε):

G2

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a$$
$$A \rightarrow B \mid S \mid \varepsilon$$
$$B \rightarrow b$$

Passo 2 (cont.) — agora eliminar a nova regra $A \rightarrow \varepsilon$. A aparece duas vezes em «ASA» (gerando SA, AS e S) — todas as combinações:

G3

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA \mid S$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b$$

(A regra $S \rightarrow S$ que surgiu será removida no passo seguinte, por ser unitária/redundante.)

Passo 3 — eliminar regras unitárias

Removemos $S \rightarrow S$ (redundante), depois $S_0 \rightarrow S$ (copiando as produções de S para S₀), depois $A \rightarrow B$ (copiando b) e $A \rightarrow S$ (copiando as produções de S para A). O resultado após todas as eliminações unitárias:

G7

$S_0 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$
 $S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$
 $A \rightarrow b \mid ASA \mid aB \mid a \mid AS \mid SA$
 $B \rightarrow b$

Passo 4 — padronizar. As regras «ASA» (três símbolos) e «aB», «SA», «AS» (com terminal ou muito longas) são ajustadas criando variáveis auxiliares A1 (para SA) e A2 (para o terminal a):

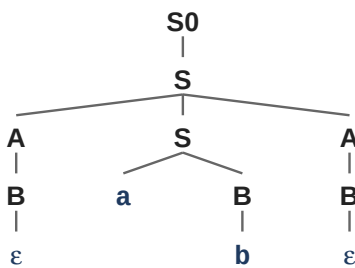
G8 (FNC final)

$S_0 \rightarrow AA1 \mid A2B \mid a \mid AS \mid SA$
 $S \rightarrow AA1 \mid A2B \mid a \mid AS \mid SA$
 $A \rightarrow b \mid AA1 \mid A2B \mid a \mid AS \mid SA$
 $A1 \rightarrow SA$
 $A2 \rightarrow a$
 $B \rightarrow b$

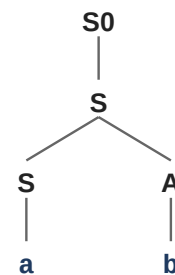
Todas as regras estão agora no formato $V \rightarrow WX$ ou $V \rightarrow a$. A gramática está na FNC. ✓

Por que a conversão preserva a linguagem?

A cada passo inserimos regras que "compensam" o que foi removido, de modo que as mesmas cadeias continuam sendo geradas. Veja a derivação da cadeia «ab» na gramática original G1 (à esquerda, com nós ϵ que serão eliminados) e na versão G3 (à direita, já sem os ϵ — a derivação simplesmente "pula" os passos que só produziam vazio):



Em G1: derivação de «ab» com nós ϵ



Em G3: mesma cadeia, sem os ϵ

4. Segundo exemplo de conversão

Convertendo a gramática de $L = \{ w \in \{a,b\}^* \mid w \text{ começa e termina com o mesmo símbolo} \}$ para a FNC. A gramática inicial (da Aula 11):

Gramática inicial

$$S \rightarrow S1 \mid S2$$
$$S1 \rightarrow a \mid aTa$$
$$S2 \rightarrow b \mid bTb$$
$$T \rightarrow Ta \mid Tb \mid \epsilon$$

Após adicionar $S0 \rightarrow S$, remover $T \rightarrow \epsilon$ (gerando aa , bb e $T \rightarrow a \mid b$), eliminar as unitárias (S , $S1$, $S2$ acabam desnecessárias) e padronizar as regras, chega-se a:

FNC final

$$S0 \rightarrow aA \mid AU \mid AA \mid bB \mid BV \mid BB$$
$$T \rightarrow TA \mid TB \mid a \mid b$$
$$U \rightarrow TA \qquad V \rightarrow TB$$
$$A \rightarrow a \qquad B \rightarrow b$$

(A regra $S0 \rightarrow aA$ vem de « aTa » após padronizar; U e V são auxiliares para « Ta » e « Tb »; A e B convertem os terminais a e b que apareciam junto de variáveis.)

5. Duas propriedades importantes da FNC

Essas propriedades são o que torna a FNC tão útil — e são a base do algoritmo CYK.

Propriedade 1 — Toda derivação tem exatamente $2n - 1$ passos

Dada uma gramática na FNC e uma cadeia $w \in L(G)$ de comprimento $n \geq 1$, a derivação de w usa **exatamente $2n - 1$ passos**.

Por quê: só há dois tipos de regra. As do tipo $V \rightarrow WX$ aumentam a cadeia intermediária em 1 símbolo — são necessárias $n - 1$ delas para chegar a n variáveis. As do tipo $V \rightarrow a$ apenas trocam variável por terminal (não mudam o tamanho) — são n delas. Total: $(n - 1) + n = 2n - 1$. Em termos de árvore: uma árvore binária com n folhas tem $n - 1$ nós internos, mais n substituições finais por terminais.

Consequência: dá para decidir se G gera w testando todas as derivações de $2n - 1$ passos. Isso *funciona*, mas tem custo **exponencial** — por isso precisamos de algo melhor (o CYK).

Propriedade 2 — Decomposição recursiva (base do CYK)

Se uma variável V deriva uma subcadeia $w_i \dots w_k$ (com $k > i$), então existe um ponto de quebra j ($i \leq j < k$) e duas variáveis V_1, V_2 tais que: G tem a regra $V \rightarrow V_1 V_2$, e $V_1 \Rightarrow^* w_i \dots w_j$ e $V_2 \Rightarrow^* w_{j+1} \dots w_k$.

Ou seja: para saber se V gera um pedaço da cadeia, basta **quebrá-lo em dois** e verificar se as duas metades são geradas por variáveis que se combinam por uma regra. Essa estrutura permite uma estratégia **bottom-up de tempo polinomial** — exatamente o algoritmo CYK da próxima aula.

Exemplo da propriedade 2: na gramática G_8 , temos $A \Rightarrow^* babab$ porque $A \rightarrow SA$, com $S \Rightarrow^* bab$ e $A \Rightarrow^* ab$. A subcadeia foi quebrada em duas partes (bab e ab), cada uma gerada por uma variável, e as duas se combinam pela regra $A \rightarrow SA$.

Exercícios resolvidos

Exercício 1 — Converta para a FNC: $S \rightarrow aSb \mid \epsilon$

Passo 1 — nova inicial:

$$\begin{aligned} S_0 &\rightarrow S \\ S &\rightarrow aSb \mid \epsilon \end{aligned}$$

Passo 2 — eliminar $S \rightarrow \epsilon$. S aparece em «aSb» (vira «ab») e em « $S_0 \rightarrow S$ » (vira $S_0 \rightarrow \epsilon$):

$$\begin{aligned} S_0 &\rightarrow S \mid \epsilon \\ S &\rightarrow aSb \mid ab \end{aligned}$$

Passo 3 — eliminar a unitária $S_0 \rightarrow S$ (copia as produções de S para S_0):

$$\begin{aligned} S_0 &\rightarrow aSb \mid ab \mid \epsilon \\ S &\rightarrow aSb \mid ab \end{aligned}$$

Passo 4 — padronizar. Criamos $A \rightarrow a$, $B \rightarrow b$, e quebramos «aSb» e «ab» em regras binárias ($X = Sb$; a auxiliar para o par):

$$\begin{aligned} S_0 &\rightarrow AX \mid AB \mid \epsilon \\ S &\rightarrow AX \mid AB \\ X &\rightarrow SB \\ A &\rightarrow a \quad B \rightarrow b \end{aligned}$$

Todas as regras estão em $V \rightarrow WX$ ou $V \rightarrow a$, e só S_0 tem a regra $\rightarrow \epsilon$. FNC completa. ✓

Exercício 2 — Por que a FNC é útil para algoritmos?

Porque as árvores sintáticas ficam **binárias** e toda derivação de uma cadeia de tamanho n tem exatamente $2n - 1$ passos (comprimento previsível). Combinada com a propriedade de decomposição recursiva (uma subcadeia é gerada por V sse puder ser quebrada em duas metades geradas por V_1 e V_2 com $V \rightarrow V_1V_2$), isso permite preencher uma tabela de baixo para cima e decidir a pertinência em tempo polinomial $O(n^3)$ — é exatamente o que o algoritmo CYK faz.

Exercício 3 — Quantos passos tem a derivação de «aabb» numa GLC na FNC?

A cadeia «aabb» tem comprimento $n = 4$. Pela Propriedade 1, a derivação tem exatamente $2n - 1 = 2 \cdot 4 - 1 = 7$ **passos**: são $n - 1 = 3$ aplicações de regras $V \rightarrow WX$ (que montam a árvore binária de 4 folhas) e $n = 4$ aplicações de $V \rightarrow a$ (que colocam os terminais nas folhas). ✓

Resumo de Estudo — Aula 13

Algoritmo CYK (Cocke–Younger–Kasami) • Seção 7.2

Na aula anterior aprendemos a colocar uma gramática na Forma Normal de Chomsky. Agora vamos usar essa forma para resolver um problema central: dada uma gramática G na FNC e uma cadeia w , **decidir se w pertence a $L(G)$** — isto é, se G consegue gerar w . O algoritmo CYK faz isso de forma eficiente, em tempo polinomial.

1. O problema e a ideia geral

Entrada: uma gramática G na FNC (variável inicial S_0) e uma cadeia $w = w_1w_2\dots w_n$. **Saída:** aceitar se $w \in L(G)$, rejeitar caso contrário.

Vimos que testar todas as derivações de $2n-1$ passos resolve o problema, mas com custo **exponencial**. O CYK evita isso usando **programação dinâmica**: decompõe o problema em subproblemas menores (as subcadeias) e reaproveita as respostas.

Casos triviais (resolvidos direto pelas propriedades da FNC)

- Se $w = \varepsilon$: G gera ε **se e somente se** existir a regra $S_0 \rightarrow \varepsilon$.
- Se $w = a$ (um único terminal): G gera « a » sse existir a regra $S_0 \rightarrow a$. (Mais geral: uma variável V deriva o terminal « a » sse G tiver a regra $V \rightarrow a$.)

Os casos interessantes são cadeias de comprimento $n \geq 2$, tratados pela tabela abaixo.

2. A estrutura de dados: a matriz M

O coração do CYK é uma **matriz triangular superior** M . A célula $M(i, j)$ guarda a **lista de variáveis de G que derivam a subcadeia $w_iw_{i+1}\dots w_j$** (todos os pedaços da cadeia que vão da posição i até a posição j).

Por exemplo, $M(4,7)$ = lista das variáveis que derivam a subcadeia $w_4w_5w_6w_7$. As células abaixo da diagonal (onde $i > j$) não são usadas, porque não existe subcadeia "para trás".

Regra de aceitação

Depois de preencher toda a matriz, a resposta é imediata: se a variável inicial $S_0 \in M(1, n)$ — ou seja, se S_0 deriva a cadeia inteira, da posição 1 à n — então **aceite**; senão, **rejeite**.

O preenchimento é feito em **ordem crescente do tamanho das subcadeias**: primeiro todas as de tamanho 1, depois as de tamanho 2, tamanho 3, ..., até a cadeia completa (a célula $M(1, n)$). É a essência do bottom-up.

3. Como preencher a matriz

Subcadeias de tamanho 1 (a diagonal principal)

Para cada posição i de 1 a n : olhe o terminal $b = w_i$ e, para toda regra $V \rightarrow b$, adicione a variável V em $M(i, i)$. (Aqui usamos a propriedade da FNC: V deriva o terminal b sse existir a regra $V \rightarrow b$.)

Subcadeias de tamanho $L \geq 2$ (o passo indutivo)

Para preencher $M(i, j)$ de uma subcadeia de tamanho L , testamos **todos os pontos de quebra k** entre i e $j-1$. Cada k divide a subcadeia em duas partes: $w_i \dots w_k$ (já calculada em $M(i, k)$) e $w_{k+1} \dots w_j$ (já calculada em $M(k+1, j)$). Então, para toda regra $V \rightarrow WX$ com $W \in M(i, k)$ e $X \in M(k+1, j)$, adicionamos V em $M(i, j)$.

Por que isso funciona (Propriedade 2 da FNC)

Lembre da aula passada: se V deriva uma subcadeia, então existe um ponto de quebra que a divide em duas metades geradas por duas variáveis W e X , com $V \rightarrow WX$. O CYK simplesmente **testa todos os pontos de quebra possíveis** e todas as regras $V \rightarrow WX$, aproveitando as células já preenchidas para as subcadeias menores.

Esquematicamente, para preencher $M(3, 6)$ testamos as três quebras possíveis: em $k=3$ ($w_3 \mid w_4 w_5 w_6$), em $k=4$ ($w_3 w_4 \mid w_5 w_6$) e em $k=5$ ($w_3 w_4 w_5 \mid w_6$), combinando as variáveis das células correspondentes através das regras $V \rightarrow WX$.

4. O algoritmo consolidado

Reunindo tudo, este é o CYK completo:

```
Entrada: G na FNC (inicial  $S_0$ ), cadeia  $w = w_1 \dots w_n$ 

1. se  $w = \varepsilon$ : aceite sse existe  $S_0 \rightarrow \varepsilon$  ; senão rejeite
2. se  $w = a$  (um terminal): aceite sse existe  $S_0 \rightarrow a$ 
3. para  $i = 1$  até  $n$ :                               # subcadeias de tamanho 1
4.      $b = w_i$ 
5.     para toda regra  $V \rightarrow b$ : adicione  $V$  em  $M(i,i)$ 
6. para  $L = 2$  até  $n$ :                               # tamanho da subcadeia
7.     para  $i = 1$  até  $n-L+1$ :                         # posição inicial
8.          $j = i + L - 1$                              # posição final
9.         para  $k = i$  até  $j-1$ :                       # ponto de quebra
10.            para toda regra  $V \rightarrow WX$  com
                 $W \in M(i,k)$  e  $X \in M(k+1,j)$ :
                adicione  $V$  em  $M(i,j)$ 
11. se  $S_0 \in M(1,n)$ : aceite ; senão rejeite
```


5. Exemplo trabalhado passo a passo

Vamos rodar o CYK para a linguagem das cadeias com **a mesma quantidade de a's e b's**, testando a cadeia **w = aabab b** (isto é, a a b a b b, $n = 6$). A gramática original e sua FNC:

Original: $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$

FNC utilizada

$S0 \rightarrow AT \mid BU \mid SS \mid AB \mid BA \mid \epsilon$

$S \rightarrow AT \mid BU \mid SS \mid AB \mid BA$

$T \rightarrow SB \qquad U \rightarrow SA$

$A \rightarrow a \qquad B \rightarrow b$

Passo 1 — diagonal (tamanho 1). Cada posição recebe a variável do seu terminal: onde há «a» entra A (regra $A \rightarrow a$); onde há «b» entra B (regra $B \rightarrow b$). Assim $M(i,i)$ fica: A, A, B, A, B, B.

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	∅	∅	∅	∅	∅
2		A	∅	∅	∅	∅
3			B	∅	∅	∅
4				A	∅	∅
5					B	∅
6						B

Matriz após o passo 1 (só a diagonal preenchida).

Passo 2 — tamanho 2. Para cada par adjacente, testamos as regras $V \rightarrow WX$. Exemplos: $M(2,3)$ recebe a subcadeia «ab» → como $M(2,2)=A$ e $M(3,3)=B$, e há a regra $S \rightarrow AB$ (e $S0 \rightarrow AB$), colocamos $\{S0, S\}$. Já $M(1,2) = \text{«aa»}$ → precisaríamos de uma regra $V \rightarrow AA$, que não existe, então fica ∅.

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
2		A	S0, S	$\emptyset$	$\emptyset$	$\emptyset$
3			B	S0, S	$\emptyset$	$\emptyset$
4				A	S0, S	$\emptyset$
5					B	$\emptyset$
6						B

Após o tamanho 2. As subcadeias «ab» (posições 2-3, 3-4) e «ba» (4-5) geram {S0,S} pelas regras $S \rightarrow AB$ e $S \rightarrow BA$. Os pares «aa» (1-2) e «bb» (5-6) ficam $\emptyset$, pois não há regra $V \rightarrow AA$ nem $V \rightarrow BB$.

Passos 3 a 5 — tamanhos 3, 4 e 5. Continuamos combinando, sempre testando todos os pontos de quebra. Vão surgindo as variáveis auxiliares U (regra $U \rightarrow SA$) e T (regra $T \rightarrow SB$) nas subcadeias maiores, e $\{S_0, S\}$ nas subcadeias balanceadas.

Passo final — tamanho 6 (a cadeia inteira). Preenchemos $M(1,6)$. Testando as quebras, encontramos uma combinação que gera S_0 . Como $S_0 \in M(1,6)$, a cadeia é **aceita**. A matriz completa fica:

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	S_0, S
2		A	S_0, S	U	S_0, S	T
3			B	S_0, S	T	$\emptyset$
4				A	S_0, S	T
5					B	$\emptyset$
6						B

Matriz M completa. A célula destacada $M(1,6)$ contém $S_0 \rightarrow$ a cadeia $aabab\ b$ é aceita. As células $\emptyset$ representam subcadeias que nenhuma variável deriva.

6. Comentários finais

Complexidade de tempo

Sejam n = comprimento da cadeia, r = número de regras, v = número de variáveis. O passo das subcadeias de tamanho 1 custa $O(r \cdot n)$. O bloco principal tem três laços aninhados (tamanho, i , k), cada um $O(n) \rightarrow O(n^3)$ iterações, e cada uma testa as regras a custo $O(r)$. Logo o total é $O(r \cdot n^3)$ — ou simplesmente $O(n^3)$ quando r é pequeno em relação a n . Muito melhor que o exponencial da força bruta.

- **Bônus 1:** a matriz M não diz só se a cadeia inteira pertence à linguagem — ela revela **todas as subcadeias** que pertencem a $L(G)$ (basta ver onde S_0 aparece). Útil para buscar padrões em cadeias de tamanho moderado.
- **Bônus 2:** esta versão é apenas um **reconhecedor** (aceita/rejeita); não constrói a árvore sintática. Para isso, cada variável inserida precisaria guardar ponteiros para as células de onde veio (as variáveis W e X que a geraram), e depois se faz uma busca a partir de $M(1,n)$.

Exercícios resolvidos

Exercício 1 — Rode o CYK para a cadeia «ab» com a FNC do exemplo

A cadeia «ab» tem $n = 2$. Passo 1 (diagonal): $M(1,1) = \{A\}$ (pois $w_1 = a$) e $M(2,2) = \{B\}$ (pois $w_2 = b$). Passo tamanho 2: $M(1,2)$ testa a única quebra $k=1$, com $W \in M(1,1) = \{A\}$ e $X \in M(2,2) = \{B\}$. Existe regra $V \rightarrow AB$? Sim: $S \rightarrow AB$ e $S0 \rightarrow AB$. Logo $M(1,2) = \{S0, S\}$.

Como $S0 \in M(1,2)$, a cadeia «ab» é **aceita** (ela tem um «a» e um «b» — quantidades iguais). ✓

Exercício 2 — Rode o CYK para «aab» (deve rejeitar)

$n = 3$. Diagonal: $M(1,1)=\{A\}$, $M(2,2)=\{A\}$, $M(3,3)=\{B\}$.

Tamanho 2: $M(1,2) = \langle aa \rangle \rightarrow$ precisa de $V \rightarrow AA$; não existe $\rightarrow \emptyset$. $M(2,3) = \langle ab \rangle \rightarrow V \rightarrow AB$ existe $\rightarrow \{S0, S\}$.

Tamanho 3: $M(1,3)$ testa $k=1$ ($A \mid M(2,3)=\{S0, S\}$) e $k=2$ ($M(1,2)=\emptyset \mid B$). Para $k=1$ precisaríamos de $V \rightarrow A \cdot S0$ ou $V \rightarrow A \cdot S$; não há regra assim. Para $k=2$ uma das partes é $\emptyset$. Logo $M(1,3) = \emptyset$.

Como $S0 \notin M(1,3)$, a cadeia «aab» é **rejeitada** — de fato ela tem dois a's e um b, quantidades diferentes. ✓

Exercício 3 — Qual a complexidade de decidir « $w \in L(G)$ » com o CYK?

$O(r \cdot n^3)$, com $n = |w|$ e r = número de regras da gramática na FNC; ou $O(n^3)$ considerando r constante. Isso vem dos três laços aninhados (tamanho, posição inicial e ponto de quebra), cada um variando $O(n)$ vezes, com o teste de regras custando $O(r)$ por célula. É polinomial — a grande vantagem sobre testar as $2n-1$ derivações por força bruta, que é exponencial. ✓

Resumo de Estudo — Aula 14

Autômato com Pilha (Parte 1) • Seção 2.2

Até agora descrevemos linguagens livres-do-contexto **gerando** cadeias, com gramáticas. Agora vamos vê-las do outro lado: com uma **máquina que reconhece** essas linguagens — o autômato com pilha. Ele é o análogo, para as LLCs, do que o autômato finito é para as linguagens regulares.

1. O que é um autômato com pilha

Um **autômato com pilha (AP)** é parecido com um autômato finito não-determinístico, mas com uma peça extra: uma **pilha**. Essa memória adicional (ilimitada, mas de acesso restrito) é o que dá ao AP o poder de reconhecer linguagens não-regulares como $0^n 1^n$.

Por que a pilha importa

- APs são **equivalentes em poder às GLCs**: toda linguagem gerada por uma GLC é reconhecida por algum AP, e vice-versa (provaremos nas próximas aulas).
- Às vezes é **mais fácil projetar um AP** do que uma GLC (e às vezes o contrário) — ter as duas ferramentas é uma vantagem.
- O AP é um **modelo matemático de computação** para as LLCs.

Relembre a pilha: é uma estrutura **LIFO** (Last-In, First-Out). Só se insere (empilha) e remove (desempilha) elementos pelo **topo**. A cada transição, o AP pode simultaneamente **ler** um símbolo da fita de entrada e **empilhar/desempilhar** um símbolo da pilha.

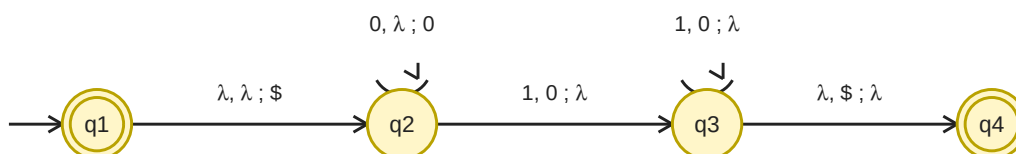
2. Exemplo motivador: $L = \{ 0^n 1^n \mid n \geq 0 \}$

A ideia é usar a pilha para "contar" os zeros e depois casá-los com os uns:

- **1.** A cada 0 lido, empilhe um 0.
- **2.** Ao ler o primeiro 1, comece a desempilhar um 0 para cada 1 lido.
- **3. Condições de parada:** se a entrada acabar exatamente quando a pilha de zeros esvaziar, **aceite**; se a pilha esvaziar antes do fim da entrada, ou se sobrar símbolo na pilha ao fim, **rejeite**.

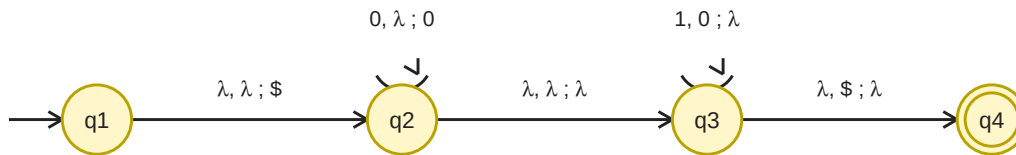
Um truque comum é empilhar primeiro um símbolo **sentinela** \$ (marca o fundo da pilha), para detectar facilmente quando ela está "vazia". A notação das transições é **a, b ; c** — significando: lê «a» da fita, desempilha «b» e empilha «c» (usamos λ para "nada").

AP1 — versão do livro



$q1 \rightarrow q2$ empilha o sentinela $\$$. O laço em $q2$ empilha um 0 para cada 0 lido. A transição $q2 \rightarrow q3$ (e o laço em $q3$) desempilham um 0 para cada 1 lido. Por fim $q3 \rightarrow q4$ remove o $\$$ e aceita. Note que $q1$ e $q4$ são de aceitação — $q1$ aceita a cadeia vazia direto.

AP2 — versão alternativa



Mesma linguagem, com a diferença de que a transição de $q2$ para $q3$ é por λ (não consome entrada nem mexe na pilha), separando claramente a fase de empilhar da fase de desempilhar.

3. Definição formal

Definição 2.13 — Autômato com Pilha

Um autômato com pilha é uma **6-upla** $(Q, \Sigma, \Gamma, \delta, q_0, F)$, onde Q, Σ, Γ e F são conjuntos finitos, e:

- Q — conjunto de estados;
- Σ — alfabeto de entrada;
- Γ (gama) — **alfabeto de pilha** (pode ser diferente de Σ !);
- δ — função de transição: $\delta : Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \rightarrow P(Q \times \Gamma_{\epsilon})$, onde $\Sigma_{\epsilon} = \Sigma \cup \{\epsilon\}$ e $\Gamma_{\epsilon} = \Gamma \cup \{\epsilon\}$. O $P(\dots)$ indica conjunto de possibilidades (não-determinismo).
- $q_0 \in Q$ — estado inicial;
- $F \subseteq Q$ — conjunto de estados de aceitação.

Como ler a função de transição: $\delta(q_i, a, b) = (q_j, c)$ significa: "estando no estado q_i , com «a» na fita e «b» no topo da pilha, vá para q_j , desempilhe «b» e empilhe «c»". Qualquer um de a, b, c pode ser ϵ (não ler / não desempilhar / não empilhar).

Descrição formal do AP1

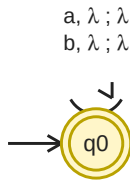
Para o AP1 do exemplo $0^n 1^n$: $Q = \{q_1, q_2, q_3, q_4\}$; $\Sigma = \{0, 1\}$; $\Gamma = \{0, \$\}$; $F = \{q_1, q_4\}$; estado inicial q_1 ; e δ dada pela tabela (entradas em branco = $\emptyset$):

$\delta(q_1, \lambda, \lambda) = \{ (q_2, \$) \}$	# empilha sentinela
$\delta(q_2, 0, \lambda) = \{ (q_2, 0) \}$	# empilha 0
$\delta(q_2, 1, 0) = \{ (q_3, \lambda) \}$	# 1º 1: desempilha 0
$\delta(q_3, 1, 0) = \{ (q_3, \lambda) \}$	# demais 1: desempilha 0
$\delta(q_3, \lambda, \$) = \{ (q_4, \lambda) \}$	# remove sentinela e aceita

4. Uma coleção de exemplos de APs

Linguagens regulares (a pilha nem é usada)

Toda linguagem regular é reconhecida por um AP — basta um autômato que nunca mexe na pilha (equivalente a um AFN). Por exemplo, para $L = \Sigma^* = \{a,b\}^*$ basta um único estado de aceitação com um laço que lê a ou b sem tocar na pilha:

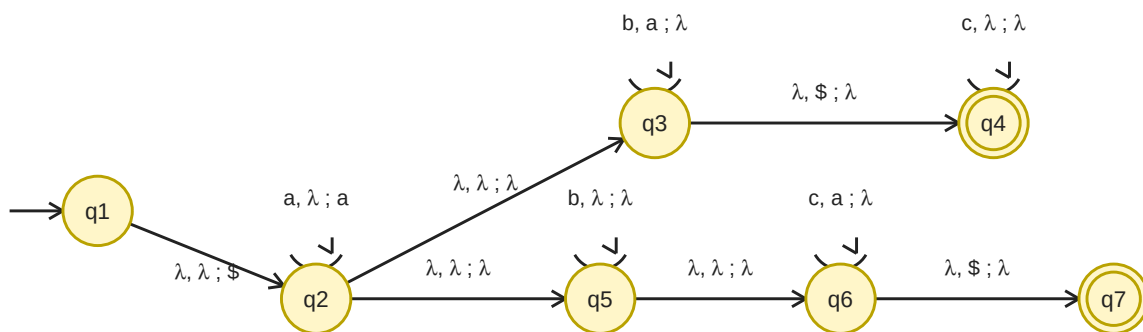


Para Σ^+ (não-vazio) usa-se dois estados: um inicial não-aceitador e, após ler o primeiro símbolo, um estado de aceitação com o mesmo laço.

Não-determinismo essencial: $L = \{ a^i b^j c^k \mid i=j \text{ ou } i=k \}$

Aqui o AP precisa "decidir", ao ver os a's, se vai compará-los com os b's ou com os c's — mas não sabe de antemão qual condição vale. A solução é o **não-determinismo**: empilha os a's e então **ramifica** em duas alternativas; se *qualquer* ramo aceitar, a cadeia é aceita.

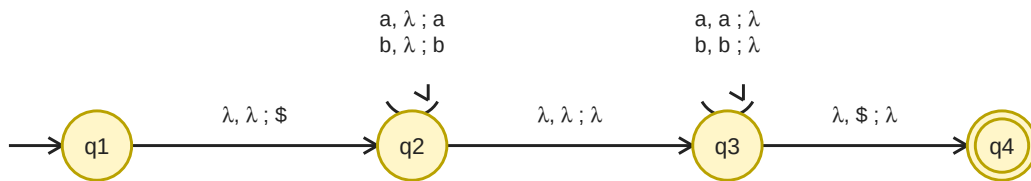
- 1. Para cada «a» lido, empilhe-o.
- 2. Não-deterministicamente, escolha um dos dois caminhos: (i) desempilhar um «a» para cada «b» lido; ou (ii) ignorar os b's e desempilhar um «a» para cada «c».
- 3. Se, no ramo escolhido, a entrada acabar com a pilha vazia, aceite.



Ramo de cima ($q2 \rightarrow q3 \rightarrow q4$): compara a's com b's ($i = j$). Ramo de baixo ($q2 \rightarrow q5 \rightarrow q6 \rightarrow q7$): lê os b's sem usar a pilha e compara a's com c's ($i = k$). O não-determinismo em $q2$ tenta os dois.

Palíndromos por reflexão: $L = \{ w w^R \mid w \in \{0,1\}^* \}$

A cadeia é a primeira metade seguida do seu reverso. Estratégia: empilhe a primeira metade; então, não-deterministicamente, "adivinha" o meio (transição λ de q_2 para q_3) e comece a casar cada símbolo lido com o topo da pilha. A GLC correspondente é $S \rightarrow aSa \mid bSb \mid \epsilon$.

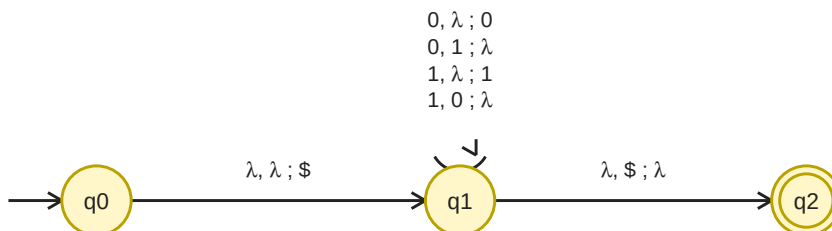


No laço de q_2 empilha-se cada símbolo lido ($a \rightarrow a$, $b \rightarrow b$). A transição λ para q_3 marca o meio. No laço de q_3 , cada símbolo lido deve casar com o topo da pilha, desempilhando. Ao encontrar o \$, aceita. Estas cadeias têm sempre comprimento par.

Cuidado — palíndromo geral $\neq w w^R$. A linguagem de *todos* os palíndromos (comprimento par **ou** ímpar) tem GLC $S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$: as opções a e b cobrem o símbolo central dos palíndromos de comprimento ímpar. No AP, isso corresponde a permitir também "adivinhar" o meio consumindo um único símbolo central.

Contagem balanceada: $L = \{ w \in \{0,1\}^* \mid w \text{ tem tantos 0's quanto 1's} \}$

GLC: $S \rightarrow 1S0 \mid 0S1 \mid SS \mid \epsilon$. No AP, a pilha guarda o "excesso" de um símbolo sobre o outro. A ideia é: ao ler um símbolo, ou ele cancela um símbolo oposto no topo da pilha, ou é empilhado (aumentando o excesso do seu tipo).



No único laço de q_1 : ler 0 empilha 0 ($0, \lambda; 0$) ou cancela um 1 do topo ($0, 1; \lambda$); ler 1 empilha 1 ($1, \lambda; 1$) ou cancela um 0 do topo ($1, 0; \lambda$). A cadeia é aceita se, ao fim, só restar o sentinela \$ (pilha "vazia"). Ex.: 1001 é aceita; 10010 é rejeitada.

Exercícios resolvidos

A aula propõe projetar APs para várias linguagens e simular computações. Resolvemos abaixo, descrevendo a estratégia de cada AP (a descrição informal é a parte que mais cai em prova).

Ex. 1(a) — $L = \{ w \in \{0,1\}^* \mid w \text{ tem pelo menos tantos 0's quanto 1's} \}$

Estratégia: use a pilha para acompanhar a diferença entre 0's e 1's. Ao ler 0, empilhe-o (ou cancele um 1); ao ler 1, cancele um 0 do topo (ou, se for possível, empilhe um marcador de "1 em excesso"). Aceite se ao final o número de 0's empilhados não for negativo em relação aos 1's — na prática, aceite se nunca faltou 0 para casar com um 1 e sobra (ou empata) 0's ao fim.

Descrição prática: empilhe \$; para cada 0, empilhe X; para cada 1, se o topo é X desempilhe (casa com um 0 anterior), senão empilhe um Y (registra 1 sem par). Aceite se, ao fim, não houver Y na pilha (todo 1 teve um 0 correspondente). ✓

Ex. 1(b) — $L = \{ 0^i 1^j \mid j \geq i \}$

Precisamos de pelo menos tantos 1's quanto 0's. Empilhe um marcador para cada 0; ao ler os 1's, desempilhe um marcador por 1 até esvaziar (casando i deles) e, depois, aceite qualquer quantidade extra de 1's sem usar a pilha.

```
q0 --(λ, λ; $)--> q1      (empilha sentinela)
q1: 0, λ ; 0              (empilha um 0 por leitura)
q1 --(λ, λ; λ)--> q2      (fim dos 0s; começa a ler 1s)
q2: 1, 0 ; λ              (casa um 1 com cada 0)
q2 --(λ, $; λ)--> q3      (esvaziou a pilha)
q3: 1, λ ; λ              (1s extras: j > i, ok)
```

q3 (aceitação) permite os 1's excedentes, garantindo $j \geq i$. ✓

Ex. 1(d) — $L = \{ 0^n 1^{2n} \mid n \geq 0 \}$

Cada 0 corresponde a **dois** 1's. Dica da própria aula: para cada 0 lido, empilhe **dois** zeros; depois desempilhe um por 1 lido.

```
q0 --(λ, λ; $)--> q1
q1: 0, λ ; 00             (empilha DOIS zeros por 0 lido)
q1 --(λ, λ; λ)--> q2
q2: 1, 0 ; λ              (desempilha um zero por 1 lido)
q2 --(λ, $; λ)--> q3      (aceita)
```

Ao empilhar 2 zeros por 0 e remover 1 por 1, a pilha só esvazia quando o nº de 1's for o dobro do nº de 0's. Ex.: 011, 001111. ✓

Ex. 3 — Simule o AP de $0^n 1^n$ sobre a cadeia 0011

Usando o AP1 (topo da pilha à esquerda):

estado	entrada restante	pilha	
q1	0011	(vazia)	
q2	0011	\$	(empilhou sentinela)
q2	011	0\$	(leu 0, empilhou 0)
q2	11	00\$	(leu 0, empilhou 0)
q3	1	0\$	(leu 1, desempilhou 0)
q3	(vazia)	\$	(leu 1, desempilhou 0)
q4	(vazia)	(vazia)	(removeu \$) → ACEITA

A entrada terminou exatamente quando a pilha esvaziou → cadeia aceita. ✓

Resumo de Estudo — Equivalência AP ↔ GLC (Parte 1)

Conversão de GLC para AP • Seção 2.2

Já vimos que gramáticas livres-do-contexto e autômatos com pilha descrevem as mesmas linguagens (as LLCs). Agora vamos **provar** essa equivalência. Como todo teorema "se e somente se", a prova tem duas direções — e esta aula cobre a primeira: mostrar que, dada qualquer GLC, existe um AP equivalente.

1. O teorema da equivalência

Teorema — AP e GLC têm o mesmo poder

Uma linguagem é livre-do-contexto **se e somente se** algum autômato com pilha a reconhece. A demonstração se divide em duas partes:

- **Direção 1 (esta aula):** se A é livre-do-contexto (gerada por uma GLC), então **algum AP a reconhece**.
- **Direção 2 (próxima aula):** se A é reconhecida por um AP, então **A é gerada por uma GLC**.

Nesta aula construímos, a partir de uma GLC G qualquer, um AP P que aceita exatamente $L(G)$. A pergunta central que P responde é: *existe uma sequência de substituições, usando as regras de G, que leva da variável inicial até a cadeia de entrada w?*

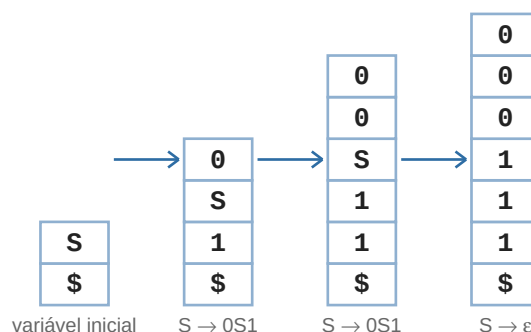
2. A ideia-chave: usar a pilha para as cadeias intermediárias

Lembre como uma GLC gera uma cadeia: partindo da variável inicial, vamos substituindo variáveis pelas regras até sobrar só terminais. O AP P vai **simular** esse processo, guardando na **pilha** a cadeia intermediária que está sendo derivada.

Vamos usar como exemplo $L = \{ 0^n 1^n \}$, com a GLC G: $S \rightarrow 0S1 \mid \epsilon$, derivando 000111:

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111$$

Uma primeira ideia ("hipotética") seria: (1) gerar toda a cadeia intermediária na pilha, aplicando as regras; (2) quando a pilha tiver só terminais, compará-los com a entrada, desempilhando quando o topo casa com a fita. Veja a fase 1 (empilhando a derivação de 000111):



Fase 1 (hipotética): a pilha acumula a cadeia intermediária. A cabeça de leitura fica parada no início da fita durante esta fase.

O problema (e a correção)

A ideia hipotética tem um defeito fatal: **não dá para acessar/substituir símbolos que não estejam no topo da pilha**. Uma variável no meio da cadeia intermediária ficaria inacessível. A correção é **mesclar as duas fases** e trabalhar sempre no topo:

Regra de operação de P (as duas situações no topo da pilha)

- **Se o topo é uma variável A:** desempilhe A e empilhe, em seu lugar, o lado direito de **uma** das regras de A (escolha não-determinística). Isso não consome entrada.
- **Se o topo é um terminal a:** compare com o símbolo atual da fita. Se forem iguais, desempilhe e avance a cabeça de leitura; se diferentes, este ramo do não-determinismo falha.

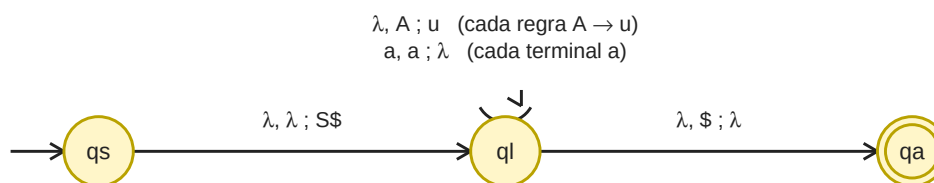
3. Construção do AP P (Lema 2.21)

Descrição informal (Sipser)

- 1. Empilhe o marcador \$ e a variável inicial S.
- 2. Repita, em cada ramo de computação possível:
 - (a) Se o topo é uma variável A: escolha não-deterministicamente uma regra $A \rightarrow u$ e empilhe u **em ordem reversa** (o primeiro símbolo de u fica no topo).
 - (b) Se o topo é um terminal a: leia o próximo símbolo da entrada; se for igual a «a», continue; senão, rejeite neste ramo.
 - (c) Se o topo é o marcador \$: entre no estado de aceitação (aceita se a entrada tiver sido totalmente lida).

Diagrama geral de P

Três estados bastam: q_{start} (inicializa), q_{loop} (faz todo o trabalho) e q_{accept} :



A aresta $\lambda, \lambda; S\$$ empilha o inicial S sobre o sentinel \$\$. No laço de q_{loop} ficam TODAS as regras (uma transição $\lambda, A; u$ por regra $A \rightarrow u$) e o casamento de terminais ($a, a; \lambda$). Quando o \$ aflora no topo, a transição $\lambda, \$; \lambda$ leva à aceitação.

Detalhe técnico: empilhar uma cadeia (transições intermediárias)

A pilha só aceita um símbolo por transição, mas uma regra $A \rightarrow u$ pode ter u com vários símbolos. A solução é quebrar isso em **transições intermediárias**, com estados auxiliares. E há um ponto crucial: **empilha-se em ordem reversa**, para que o primeiro símbolo de u termine no topo. Por exemplo, para a regra que substitui s por «xyz» ao ler «a»:

Uma transição «a, $s \rightarrow xyz$ » vira uma cadeia de transições:

$q \xrightarrow{(a, s \rightarrow z)} q_1 \xrightarrow{(\lambda, \lambda \rightarrow y)} q_2 \xrightarrow{(\lambda, \lambda \rightarrow x)} r$

Resultado na pilha (topo em cima): x / y / z (x no topo)

Empilhamos z primeiro (vai para o fundo), depois y, e por fim x (fica no topo). Assim, ao lermos a cadeia da esquerda para a direita, o topo apresenta os símbolos na ordem correta x, y, z.

4. Exemplo completo de conversão

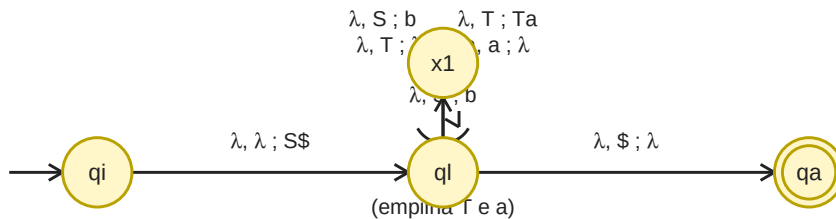
Convertendo a GLC abaixo em um AP equivalente:

GLC

$$S \rightarrow aTb \mid b$$

$$T \rightarrow Ta \mid \varepsilon$$

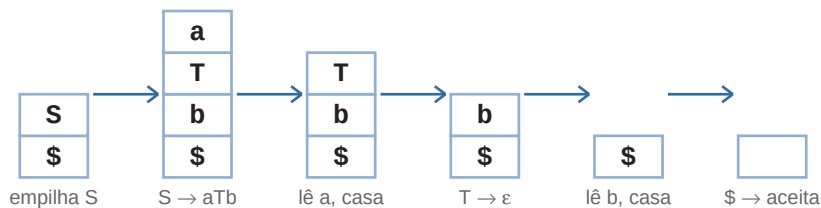
O AP tem $q_{\text{início}}$, $q_{\text{laço}}$, q_{aceita} e alguns estados intermediários para empilhar as cadeias de mais de um símbolo (como «aTb», empilhada como b, depois T, depois a):



As transições do laço de $q_{\text{laço}}$ tratam as regras diretas ($S \rightarrow b$, $T \rightarrow \lambda$) e o casamento de terminais ($a, a; \lambda$ e $b, b; \lambda$). Para a regra $S \rightarrow aTb$, que tem três símbolos, empilha-se b, depois T, depois a (ordem reversa) usando um estado intermediário; o mesmo vale para $T \rightarrow Ta$. Por clareza, a simulação abaixo mostra o efeito na pilha, que é o que realmente importa para o entendimento.

Simulando P sobre a entrada «ab»

A cadeia «ab» é gerada por $S \Rightarrow aTb \Rightarrow ab$ (usando $T \rightarrow \varepsilon$). Veja como a pilha de P reproduz essa derivação (topo em cima):



Cada passo ou expande a variável do topo (por uma regra) ou casa um terminal do topo com a fita. Ao aflorar o \$ com a entrada esgotada, P aceita «ab». ✓

Exercícios resolvidos

Exercício 1 — Converta $G: S \rightarrow 0S1 \mid \varepsilon$ em um AP

Aplicamos a construção do Lema 2.21. Três estados principais mais um intermediário para empilhar «0S1» (três símbolos):

```
q_início --( $\lambda, \lambda \rightarrow S\$$ )--> q_laço
```

No laço q_laço:

$\lambda, S \rightarrow 1$ (início da regra $S \rightarrow 0S1$: empilha 1)
seguido de $\lambda, \lambda \rightarrow S$ e $\lambda, \lambda \rightarrow 0$ (ordem reversa: topo fica 0)

$\lambda, S \rightarrow \lambda$ (regra $S \rightarrow \varepsilon$)

$0, 0 \rightarrow \lambda$ (casa terminal 0)

$1, 1 \rightarrow \lambda$ (casa terminal 1)

```
q_laço --( $\lambda, \$ \rightarrow \lambda$ )--> q_aceita
```

A regra $S \rightarrow 0S1$ é empilhada como 1 (fundo), S (meio), 0 (topo), de modo que a leitura processe primeiro o 0, depois expanda S, e por fim case o 1. ✓

Exercício 2 — Por que empilhar em ordem reversa?

Porque a pilha é LIFO: o último símbolo empilhado é o primeiro a ser lido (fica no topo). Se a regra é $A \rightarrow xyz$ e queremos que a derivação continue processando x, depois y, depois z (a ordem natural de leitura da cadeia), precisamos que x esteja no topo. Logo empilhamos na ordem inversa: z primeiro (vai ao fundo), y, e x por último (fica no topo). Se empilhássemos na ordem direta, leríamos a cadeia invertida (zyx), o que estaria errado. ✓

Exercício 3 — Simule o AP de $S \rightarrow 0S1 \mid \epsilon$ sobre a entrada «0011»

A derivação em G é $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$. A pilha de P (topo em cima) acompanha:

pilha (topo→fundo)	ação
S \$	empilha inicial
0 S 1 \$	$S \rightarrow 0S1$
S 1 \$	lê 0, casa com topo
0 S 1 1 \$	$S \rightarrow 0S1$
S 1 1 \$	lê 0, casa com topo
1 1 \$	$S \rightarrow \epsilon$
1 \$	lê 1, casa com topo
\$	lê 1, casa com topo
(vazia)	$\lambda, \$ \rightarrow \lambda \rightarrow$ ACEITA

A entrada «0011» é consumida exatamente quando o \$ aflora $\rightarrow$ aceita. ✓

Resumo de Estudo — Equivalência AP ↔ GLC (Parte 2)

Conversão de AP para GLC • Seção 2.2

Na aula anterior provamos que toda GLC tem um AP equivalente. Agora fechamos a equivalência com a **direção oposta**: dado qualquer autômato com pilha, construir uma GLC que gera exatamente a mesma linguagem. Esta é a parte mais abstrata, mas a ideia central é elegante.

Lema 2.27

Se um autômato com pilha reconhece alguma linguagem, então essa linguagem é livre-do-contexto. Juntando com o Lema 2.21 (aula anterior), concluímos: uma linguagem é livre-do-contexto **se e somente se** algum AP a reconhece.

1. Primeiro, padronizamos o AP

Para facilitar a construção, assumimos que o AP tem três propriedades. Qualquer AP pode ser facilmente adaptado para satisfazê-las:

Três requisitos sobre o AP

- 1. Tem um **único estado de aceitação**, q_{aceita} .
- 2. **Esvazia a pilha antes de aceitar** (a pilha está vazia ao aceitar).
- 3. Cada transição ou **empilha** um símbolo, ou **desempilha** um símbolo — **nunca as duas coisas ao mesmo tempo**, e nunca nenhuma das duas.

Como garantir o requisito 3 (o mais técnico)

Se alguma transição faz empilhamento e desempilhamento simultâneos, nós a "desdobramos" em duas, passando por um estado auxiliar:

```
Transição    p_i --(a, b → c)--> p_j
vira duas:   p_i --(a, b → λ)--> p_aux --(λ, λ → c)-->
p_j
```

A primeira só desempilha (b), a segunda só empilha (c). Transições que não mexem na pilha também são ajustadas de forma análoga (empilha-desempilha um símbolo "fictício").

2. A ideia central: variáveis A_{pq}

Queremos uma GLC G tal que G gere uma cadeia x **exatamente quando** x leva o AP do estado inicial ao estado de aceitação. A grande ideia é criar uma variável para **cada par de estados**:

Definição das variáveis

Para cada par de estados (p, q) , criamos uma variável A_{pq} . Ela deve gerar **todas as cadeias x que levam o AP do estado p (com a pilha vazia) ao estado q (com a pilha vazia)**.

A variável inicial da gramática é $A_{(q_0, q_{aceita})}$ — as cadeias que levam do estado inicial ao de aceitação, começando e terminando com a pilha vazia. É exatamente a linguagem do AP.

Por que "pilha vazia no início e no fim"? Graças ao requisito 2, o AP esvazia a pilha antes de aceitar; e ele começa com a pilha vazia. Então a computação inteira (do inicial ao de aceitação) é justamente um trajeto "de pilha vazia a pilha vazia" — o que a variável inicial $A_{(q_0, q_{aceita})}$ descreve.

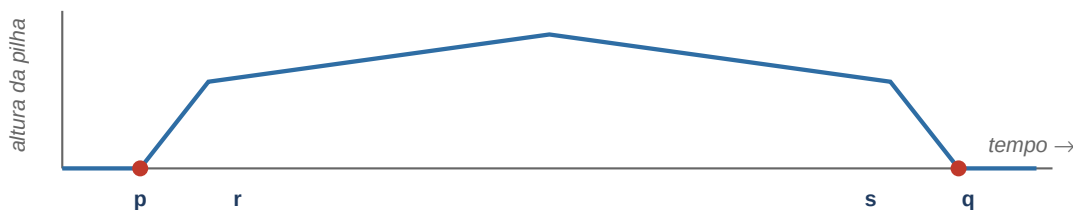
3. As duas situações de um trajeto $p \rightarrow q$

Considere um trajeto do AP de p a q , começando e terminando com a pilha vazia, lendo a cadeia x . Pense em como a **altura da pilha** evolui ao longo do tempo. Como o AP sai do vazio e volta ao vazio, há exatamente **duas situações possíveis**:

Caso 1 — a pilha só volta a ficar vazia no final

Durante todo o trajeto (exceto no início e no fim) a pilha permanece não-vazia. Isso significa que o **primeiro símbolo empilhado só é desempilhado no último passo**.

Caso 1: pilha vazia só nas pontas



O primeiro movimento (lendo «a») empilha um símbolo t e vai de p para r ; o último movimento (lendo «b») desempilha esse mesmo t e vai de s para q . No meio, o AP vai de r a s sem nunca esvaziar a pilha. Isso dá a regra:

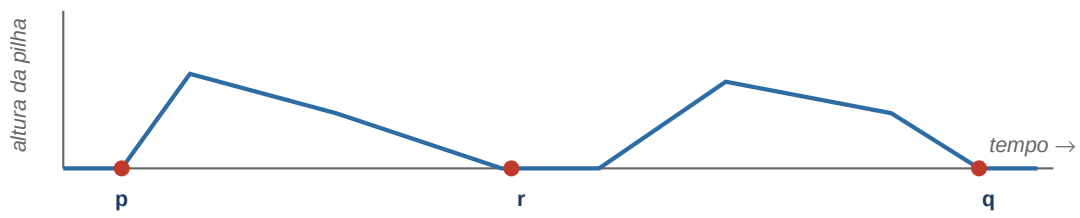
$$A_{pq} \rightarrow a A_{rs} b$$

onde $x = a \cdot y \cdot b$, e y é a subcadeia que leva de r a s (num nível de pilha acima do t). Formalmente: se $\delta(p, a, \epsilon)$ contém (r, t) e $\delta(s, b, t)$ contém (q, ϵ) , então $A_{pq} \rightarrow a A_{rs} b$ está em G .

Caso 2 — a pilha fica vazia em algum ponto do meio

Existe um estado intermediário r onde a pilha volta a ficar vazia antes de chegar em q . Nesse caso, podemos **quebrar o trajeto em dois** pedaços independentes: de p a r , e de r a q , ambos "de pilha vazia a pilha vazia".

Caso 2: pilha esvazia num ponto intermediário r



A cadeia se divide como $x = y \cdot z$, onde y leva de p a r e z leva de r a q. Isso dá a regra:

$$A_{pq} \rightarrow A_{pr} A_{rq}$$

4. A construção formal da gramática

Reunindo os dois casos, mais o caso base, definimos as regras de G . Seja $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{aceita}\})$. As variáveis de G são $\{A_{pq} \mid p, q \in Q\}$, e a variável inicial é $A_{(q_0, q_{aceita})}$.

As regras de G

- **Caso 1:** para cada $p, q, r, s \in Q$, cada $t \in \Gamma$ e cada $a, b \in \Sigma_\epsilon$, se $\delta(p, a, \epsilon)$ contém (r, t) e $\delta(s, b, t)$ contém (q, ϵ) , então adicione a regra $A_{pq} \rightarrow a A_{rs} b$.
- **Caso 2:** para cada $p, q, r \in Q$, adicione a regra $A_{pq} \rightarrow A_{pr} A_{rq}$.
- **Caso base:** para cada $p \in Q$, adicione a regra $A_{pp} \rightarrow \epsilon$ (a cadeia vazia leva um estado a si mesmo com a pilha vazia).

Intuição das regras

- A regra do Caso 1 ($A_{pq} \rightarrow a A_{rs} b$) reconhece que os símbolos das pontas «a» e «b» estão "casados" pela pilha — o t empilhado no início é o t desempilhado no fim, como um par de parênteses correspondentes.
- A regra do Caso 2 ($A_{pq} \rightarrow A_{pr} A_{rq}$) é uma simples concatenação de dois trajetos independentes emendados no estado r .
- A regra base ($A_{pp} \rightarrow \epsilon$) encerra a recursão: um trajeto de tamanho zero.

5. Por que a construção funciona (esboço da prova)

É preciso provar a afirmação-chave nas duas direções (é um "se e somente se"):

Afirmação 2.31

A_{pq} gera a cadeia x se e somente se x leva o AP de p (pilha vazia) a q (pilha vazia).

Direção «gera $\Rightarrow$ leva»

Por indução no número de passos da derivação de $A_{pq} \Rightarrow^* x$. No caso base, a derivação tem um passo ($A_{pp} \rightarrow \epsilon$), e de fato a cadeia vazia leva p a p sem mexer na pilha. No passo indutivo, a primeira regra usada é do Caso 1 ou do Caso 2, e a hipótese de indução se aplica às subderivações (A_{rs} ou A_{pr}, A_{rq}), reconstruindo o trajeto no AP.

Direção «leva $\Rightarrow$ gera»

Por indução no número de passos da computação do AP. Assuma verdadeiro para computações de comprimento $\leq k$ e prove para $k + 1$. Se x leva p a q em $k+1$ passos (pilhas vazias nas pontas), analisamos os dois casos pela forma como a pilha se comporta:

- **Caso 1** (pilha vazia só nas pontas): o primeiro símbolo empilhado t é o último a sair. Sejam r o 2º estado e s o penúltimo, e a, b os símbolos lidos no 1º e último movimentos. Então $\delta(p, a, \epsilon) \ni (r, t)$ e $\delta(s, b, t) \ni (q, \epsilon)$, logo $A_{pq} \rightarrow a A_{rs} b$ está em G . A subcadeia y (com $x = a \cdot y \cdot b$) leva r a s em $k-1$ passos sem tocar em t ; pela hipótese de indução, $A_{rs} \Rightarrow^* y$. Logo $A_{pq} \Rightarrow^* x$.
- **Caso 2** (pilha esvazia num ponto intermediário r): cada trecho (de p a r e de r a q) tem no máximo k passos, e $x = y \cdot z$. Pela hipótese de indução, $A_{pr} \Rightarrow^* y$ e $A_{rq} \Rightarrow^* z$. Como $A_{pq} \rightarrow A_{pr} A_{rq}$ está em G , temos $A_{pq} \Rightarrow^* x$.

Provadas as duas direções, a variável inicial $A(q_0, q_{aceita})$ gera exatamente as cadeias aceitas pelo AP — ou seja, $L(G) = L(P)$. ■

Conclusão dos dois lemas — o teorema completo

Lema 2.21 (Parte 1): toda GLC tem um AP equivalente. Lema 2.27 (Parte 2): todo AP tem uma GLC equivalente. Juntos, eles estabelecem que **GLCs e APs descrevem exatamente a mesma classe de linguagens — as linguagens livres-do-contexto**. Podemos, portanto, escolher a ferramenta mais conveniente (gerar com GLC ou reconhecer com AP) para cada problema.

Exercícios resolvidos

Exercício 1 — Quais são as três formas de regra na construção $AP \rightarrow GLC$?

São exatamente três tipos:

- $A_{pq} \rightarrow a A_{rs} b$ (Caso 1): quando um símbolo empilhado no primeiro movimento é desempilhado só no último — os terminais das pontas ficam "casados".
- $A_{pq} \rightarrow A_{pr} A_{rq}$ (Caso 2): quando a pilha esvazia num estado intermediário r , dividindo o trajeto em dois.
- $A_{pp} \rightarrow \varepsilon$ (caso base): trajeto vazio de um estado para si mesmo.

A variável inicial é $A_{(q_0, q_{aceita})}$. ✓

Exercício 2 — Por que exigimos que o AP esvazie a pilha antes de aceitar?

Porque a variável inicial $A_{(q_0, q_{aceita})}$ foi definida para gerar as cadeias que levam do estado inicial ao de aceitação **com a pilha vazia nas duas pontas**. Se o AP pudesse aceitar com a pilha não-vazia, essas computações não seriam capturadas por $A_{(q_0, q_{aceita})}$, e a gramática geraria a linguagem errada. Forçar o esvaziamento (requisito 2) faz a definição de A_{pq} "de pilha vazia a pilha vazia" coincidir exatamente com a condição de aceitação. ✓

Exercício 3 — Por que separar empilhar de desempilhar (requisito 3)?

A construção das regras se apoia em identificar, em cada movimento, se ele **empilha** ou **desempilha** um símbolo — isso é o que permite reconhecer o "casamento" do Caso 1 (um símbolo empilhado no início e retirado no fim) e o esvaziamento do Caso 2. Se uma transição fizesse as duas operações ao mesmo tempo, não seria possível rastrear de forma limpa a evolução da altura da pilha. Por isso desdobramos cada transição mista em duas (uma só de desempilhar, outra só de empilhar), passando por um estado auxiliar. ✓

Exercício 4 — Interprete a regra $A_{pq} \rightarrow a A_{rs} b$ como "parênteses"

Pense no símbolo empilhado t como um par de parênteses: « a » abre (empilha t) e « b » fecha (desempilha t). Tudo o que acontece entre eles — a subcadeia y gerada por A_{rs} — ocorre "dentro" desse par, num nível de pilha acima de t . É a mesma estrutura recursiva que vimos em gramáticas como $S \rightarrow aSb$ (a linguagem $a^n b^n$) ou nas expressões parentizadas: um delimitador de abertura casado com um de fechamento, com conteúdo aninhado no meio. Essa correspondência é a razão de o AP e a GLC descreverem a mesma linguagem. ✓